

Monitoring the Software Development Process with Process Mining

Saimir Bala and Jan Mendling

Vienna University of Economics and Business, Austria
`firstname.lastname@wu.ac.at`

Abstract. Software projects typically need to be monitored in detail regarding when what was done in order to demonstrate adherence to methodologies, rules, regulations, guidelines or best practices. To this end, it is of utmost importance to obtain factual knowledge from empirical evidence about the actual software development process. A major problem in this context is the lack of a centralized control of by a central system. Although it is hard to obtain full knowledge of the overall software development process, several cues can be gathered by analyzing pieces of information that are stored by supporting IT systems (e.g., issue trackers and version control). This position paper presents research in progress for extracting process knowledge from the historical data of software artifacts. This work extends the applicability of process mining techniques to software processes.

Keywords: mining software development, process mining, mining software repositories, project mining, artifacts evolution

1 Introduction

Monitoring the process of software development is highly important to project managers who want to deliver good software in time and within budget. In practice, various project management approaches exist that guide the managers during the development of software projects. Usually these guidelines and methodologies stem from experience. However, in practical scenarios, every software development endeavor is different. Therefore, there is the need to extract the real software process from the data. A starting point is the empirical evidence given from the historical evolution of the artifact. This provides important cues for monitoring the adherence of actual work to existing projects plans and milestones.

Literature has addressed several aspects about getting process insights from historical data. However, the problem has been tackled separately from different disciplines. Contributions from the process mining discipline focus on obtaining process models from well structured event logs [1]. Contributions from the mining software repositories area focus on obtaining results from a software engineering point of view, e.g., code quality, code complexity, user analysis, functional dependencies, software visualization, etc [16]. Lastly, text mining has no notion of process, but it is a fundamental field of study that helps dealing with unstructured data, such as user comments in a software repository [3].

Process mining techniques fall short when it comes to mining the overarching software process. Nevertheless, several pieces of traces of the software process can be found in system logs. Therefore, our driving question can be stated as follows. **RQ:** *What can be discovered from software development data for obtaining knowledge about the process?* To answer the question, we consider extending the process mining discipline towards software development. As a result, this work aims at assisting project managers in decision making. This is done by extracting knowledge from data that is generated during the lifetime of software projects. Unstructured data from the tools (e.g. user comments from Version Control System (VCS) and Issue Tracking System (ITS)) are also taken into account. The expected result is a body of techniques and tools that can be used to for analyzing software data in a process mining fashion. In this way, we create bridge between the research areas of process mining and mining software repositories.

The rest of the paper is organized as follows. Section 2 presents the problem and related literature, and draws the requirements for a solution. Section 3 provides the challenges and how they can be tackled. Section 4 draw the conclusions and provides a plan to address these challenges.

2 Background

2.1 Problem Definition

Software development processes are characterized as follows. First, although there is a planning phase, they are conducted creatively, i.e. there exists no strict process model that is followed by the developers. Second, they follow guidelines and methodologies on software development project, such Rational Unified Process (RUP), Scrum¹, Waterfall, etc. These guidelines are recommendations that stem from experience. However, none of them is extracted from empirical evidence. Third, typically there is a plethora of tools used and process evidence is scattered among different logs. Fourth, there is typically no central system to control the overarching process execution. Fifth, it is highly relevant to track the process because of project managers want to be able to efficiently detect performance measures such as number of issues, bottlenecks, timeliness, handover of work, productivity, etc.

Software development processes fall into the category of *project-oriented business processes* [6]. That is, ad-hoc plans performed with limited resources and time, but with a clear goal: develop a software artifact. Unlike classic business processes which are best captured with notations such as Petri nets and BPMN, software development processes fall into the category of *project-oriented business processes*, which are usually captured by models such as Gantt and PERT diagrams.

An example of software development is the following. A new version of a software needs to be developed by company X. In general, company X knows how to develop software. However, before running straightly into the development

¹ <https://www.scrum.org>

phase, the company first makes sure to gather and properly formulate all the requirements. In a Scrum scenario these requirements would be written down as user stories. At a later stage, the project manager needs to plan the time and resources allocated to the respond to project deadlines. He can go through the list of requirements that need to be implemented and assign to an effort estimation value to each of them. With the plan done, the development phase can commence. During the development phase, resources address the task in a creative way, choosing the order of the tasks according to their own knowledge and expertise, until eventually all the tasks are terminated.

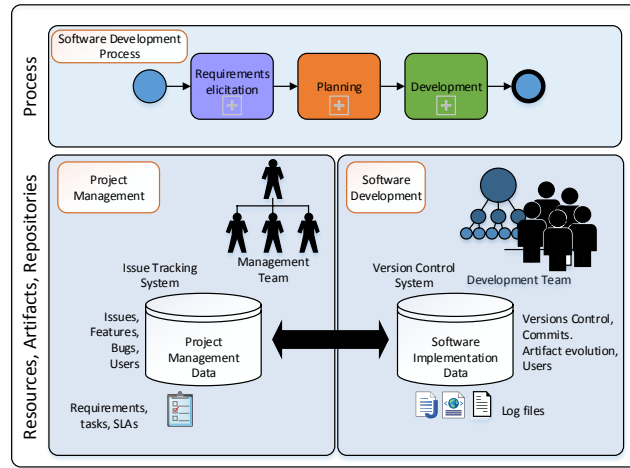


Fig. 1: Problem context

Several tools are used by software project participants to support their work. Therefore, traces about the overall process are typically scattered among different repositories and different artifacts, e.g., spreadsheets, word processor documents, programming languages files, emails, etc. This makes it difficult to obtain full knowledge about the overall business process. Two major challenges are to *correlate* heterogeneous events from different sources of evidence pertaining the same process and to *extract* information from unstructured data such as user comments when working on tasks. Thus, obtaining full process knowledge is a complex task.

Although the data is scattered and often unstructured, there are still repositories that we can study in order to obtain process knowledge. An important dimension of the software development process is the work perspective. It is particularly interesting for project managers to know if planning phase was realistic with respect to the development efforts. Two software tools that are commonly used in software development are ITS (e.g., JIRA, GitHub Issues) and VCS (e.g. Subversion, Git). The evolution of these two repositories can be analyzed to extract relevant knowledge about the software development process. Figure 1 illustrates the problem context.

Existing software repositories allow for many ways to access their log files. However, the most relevant information that they provide are about actions done by the user to change the repository state. In the more specific case, VCS logs consist of an ordered set of *commits* bearing information about users, files, timestamp, comments and type of change that were stored at particular moment in time. ITS typically come with richer information, most importantly they inform about users, task, type of task (e.g., bug, new feature, requirement, etc), timestamps, related issues, etc. To extract knowledge about the software development process, these properties of repositories must be taken into account. Table 1 gives an overview of the main information that can be found in systems, such a VCS.

Table 1: An excerpt of a VCS log data

Id	Resource	Date	Comment	Diff
1	John	2017-01-31 12:16:30	Create readme file	diff -git a/README.md b/README.md @@ -0,0 +1 @@ +# StoryMiningSoftwareRepositories
2	Mary	2017-02-01 10:13:51	Add a license	diff -git a/README b/README @@ -1,0 +2,3 @@ +The MIT License (MIT) + +Copyright (c) 2015 Mary+
3	Paul	2017-02-02 16:10:22	Updated the requirements.	diff -git a/README.md b/README.md @@ -1,4 +1,5 @@ + # string 1, string 2, string 3 diff -git a/requirements.txt b/requirements.txt @@ -0,0 +1 @@ +The software must solve the problems diff -git a/model.java b/model.java @@ -1,9 +1,10 @@ +public static methodA(){int newVal=0; @@ -21,10 +23,11 @@ + "1/0","0/0",
4	Paul	2017-02-02 15:00:02	Implement new requirements	diff -git a/test.java b/test.java @@ -0,0 +1,2 @@ +//test method A +testMethodA()

Because the project managers can benefit from a process view to better analyze hidden aspects (such as the behavioral one) of the software development process, we focus on adapting and transforming the data to fit process mining. Thus, we can better define our problem based on the four aspects of process mining [1]: *i)* time perspective; *ii)* control-flow perspective; *iii)* organizational perspective; and *iv)* case perspective. In the following, we will define four requirements, each addressing one process mining perspective.

2.2 Related Literature

This problem is related to three areas: *i)* Process Mining (PM); *ii)* Mining Software Repositories (MSR); and *iii)* Text Mining (TM).

Process mining contributions have focused on transforming this problem into a process mining problem. These approaches enrich VCS log data with case and activity information and consequently use process mining to discover a model. In this category, Kindler et al. [11,12] can discover a Petri net from a structured and enriched version control log. This approach was further improved by Rubin et al. [19] and a ProM² was provided. Poncin et al. [17] provide the FRASR framework for preprocessing software repositories such that they can be used in ProM. While providing interesting insights, these contributions leave out many important aspects of software development projects, such as for instance trying to understand whether the process was done according to the plan.

MSR focuses on software engineering aspects, like code quality metrics, modularization, code complexity, user networks, and other important metric. In general, methods from MSR provide useful dependency analyses of the repository structure. Contributions in this area focus on the users, the artifacts and the repository evolution [23,9], and network analysis of file dependency graph based on commit distance [2,22]. Also techniques for trend analysis [20] and inter-dependencies between developers [14] are proposed. However, none of these works aims at extracting knowledge about the business process.

TM focuses on obtaining structured information from unstructured textual data, and mainly uses Natural Language Processing (NLP). Works that use NLP can be found in both PM and MSR. In the Business Process Management (BPM) area, NLP techniques have been used to understand process activities [13,15] and analyze software processes under a knowledge-intensive perspective [10,18]. Likewise, in the MSR area, NLP has been used as an information extraction tool to obtain informative metrics from a software engineering perspective [21,8].

This paper combines ideas from the above mentioned areas to devise algorithms for mining the software development process by exploiting techniques from MSR.

2.3 Research Question and Solution Requirements

In the light of the above considerations, we derive the following requirements for extending process mining towards the analysis of software repositories.

R1. Mining the time perspective. Given a software repository, extract information about the temporal order of the activities. For example, the development activity takes 2 weeks on average, the average time of task creation is 15 minutes, etc.

R2. Mining the case perspective. Given a software repository, extract information about the case perspective. For example, all the bugs are solved in a 3 steps iteration, or a quality piece of code takes a conversation with 3 people and is successfully merged into the main branch after 1 week, etc.

R3. Mining the organizational perspective. Given a software repository, extract information about the organizational perspective. For example, the

² <http://www.promtools.org>

software development is carried out by a team of 4 people, the actual user roles of the company are developer and tester, etc.

R4. Mining the control-flow perspective. Given a software repository, extract information about the control-flow perspective. For example, the testing is always done before development, or while new features are worked on, also new requirements are created, etc.

3 Extracting Knowledge on Software Development

This section presents the approach for addressing the requirements. In the following, we describe approaches and challenges to address the four requirements.

3.1 Mining the Time Perspective

Mining the time perspective from a software repository is defined as extracting temporal process knowledge. This problem relates to monitoring whether the process was executed respecting a predefined plan. Typically the process involves various actors which track their work through the use of VCS. Often, the plans are not represented in standard process notation. Rather, Gantt or PERT charts are used.

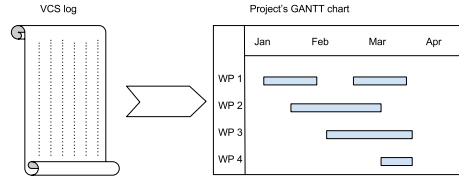


Fig. 2: Mining the time perspective of software development into a Gantt chart

Challenges are *i) time approximation* (i.e., when the activity really started, compared to when it was registered in the log); *ii) granularity* (i.e., be able to switch from a detailed view of the single events and a coarse-grained view of the overall project); and *iii) coverage* (i.e., how much work effort was put within the duration of the activity). Figure 2 depicts the idea of mining a Gantt chart from VCS logs. The output is presented in a way that is informative to project managers. Details about the technique can be found in [6].

3.2 Mining the Case Perspective

The identification of process cases is not an easy task when it comes to software development data. However, a highly informative case candidate can be considered the data artifact itself. According to [1], cases can also be characterized by the values of data elements. In line with this, it is possible to devise techniques

that study the *artifact evolution*. Although it can be input to process mining techniques, this evolution can also be analyzed as a time series. An example of such evolution is given in Figure 3, which shows the lines of code (LOC) changes over time.

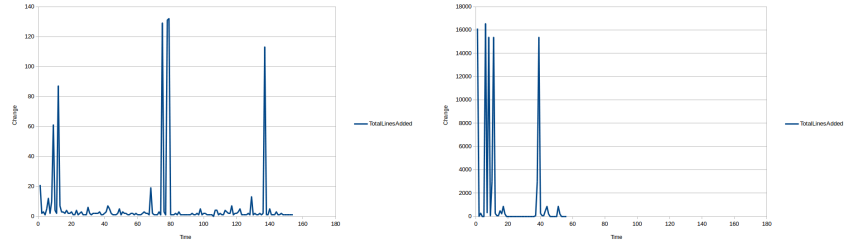


Fig. 3: Evolution of files in a version control systems: LOC changes over time

Challenges related to the artifact evolution pertain *prediction* of plateaus and *pattern recognition*. The results reflect work patterns over the artifact. For example, when a data file is not being modified anymore, its time series has a plateau and it can be interpreted that the document is now ready for release. A relevant problem in software development is bad modularization. Especially, the dependency of two artifacts on one another is considered a bad practice. Two time series can be compared together and file dependencies can be found that reflect work coupling. This is equivalent to finding dependence between two processes which might have been designed for different purposes.

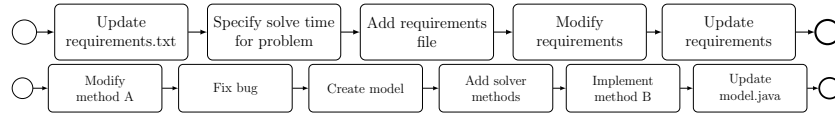


Fig. 4: Processes of two work-dependent files

We have devised a proof-of-concept in [7]. Figure 4 shows a possible outcome of the technique. The result is obtained by identifying couples of files with similar evolution and mining a business process from the comments. As the result is aggregated, several comments have been combined together and mined as a story. Among other things, the technique allows to profile existing software development projects.

3.3 Mining the Organizational Perspective

Software development projects are knowledge intensive, thus involve creativity and flexibility. Project participants are free to tackle development tasks according to their expertise and ideas to solve ad-hoc problems. De facto, members may

behave according several roles in the organization. Therefore, the discovery of the roles of a software project may give important insights on the actual project. Role discovery can help in the monitoring existing projects in two ways. First, emerging roles can be analyzed in order to have a better skills profiling of the resources. Second, emerging roles can be used to check whether resources are breaking contractual agreements or norms.

We have developed an approach for tackling the problem of mining the organizational perspective in software development projects. The approach provides information about the actors involved in the business process and their relations. In substance, it provides automatic classification of resources into company roles (e.g., developer, tester, etc) based on the comments of project participants. It works on VCS logs and provides information about the people, their roles, other systems involved, the organization hierarchy, the social network, and resource profiling. Figure 5 shows an example of resource profiles automatically inferred from VCS comments.

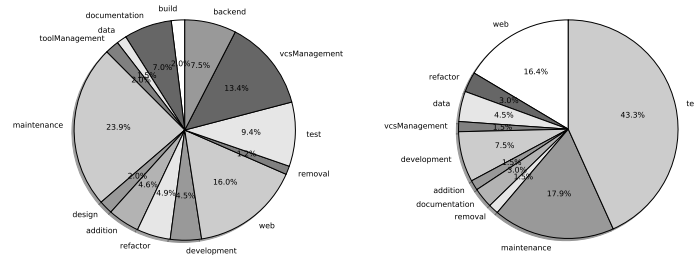


Fig. 5: Profiles of a developer and a tester, from left to right.

Challenges of using NLP concern the *jargon* used by software developers. They often use technical terms, very short phrases, or a number of codes and hyperlinks. These makes NLP techniques score low results even on simple tasks like sentence parsing, or name-entity recognition. Therefore, approaches should take into account several factors when mining for the organizational perspective. The result allows the manager to have measurable information about the resources. For example, do two people work better together and how can we build the best team? Details of the approach to extract resource profiles can be found in [4].

3.4 Mining the Control-Flow Perspective

Important knowledge of the control flow of a software development process can be extracted from VCS and ITS. One important mechanism widely used in software development for collaboration are *pull requests*. A pull request is issued every time a developer wants to contribute to the main source with a new piece of code. Pull requests trigger communication among different people and may help shedding light into problems, learn new things and obtain new ideas on the

existing software. It is interesting for project managers and developers to better understand factors and behavior that makes a pull request successful.

In ongoing work, we are investigating the relation of the pull request process and surrounding factors such as the generation of new ideas or change of behavior. We have obtained a data set of pull requests spanning over two years from an real world repository. User conversations have been manually annotated with codes that classify them into categories of comments. We are current able to discriminate whether a comment is a new idea, an assumption, a merge that closes a pull request, etc. Considering the pull request identifiers as cases and the codes as activities, we are able extract the process of the pull requests. It is interesting to compare how the conversation (i.e., behavioral perspective) unfolds before and after a pull request is merged. We plan to explain these differences by also taking into account further information from text such as emotions.

The challenge of mining the control-flow usually pertains the completeness of data and the case and activity identification. While PM contribution have been focusing mostly on the control-flow perspective, techniques from MSR hardly go beyond mining the bug lifecycle [5]. The nature of software development process makes it difficult to recognize recurrent activities when only VCS are analyzed.

4 Expected Contribution and Implications for Research

This position paper presents research on mining the software development process. Approaches towards discovering the perspectives of time, resources and cases have been developed and the results suggest that obtaining process knowledge from software repositories brings new insights for the project managers [4,6,7]. In future work, we plan to complete the research by tackling the problem of discovering the flow-perspective of the software development process. Usefulness will be evaluated through user studies.

This research extends the process mining field towards its adoption on software repositories. Project managers would benefit from this research by having a process perspective on their ongoing projects through visual models and diagrams, thus overcoming existing flat approaches based on simple indicators such as burndown charts or change plots offered by existing tools.

References

1. van der Aalst, W.M.P.: Process mining: data science in action. Springer (2016)
2. Abate, P., Boender, J., Di Cosmo, R., Zacchiroli, S.: Strong dependencies between software components. 2009 3rd Int. Symp. Empir. Softw. Eng. Meas. ESEM 2009 pp. 89–99 (2009)
3. Aggarwal, C., Zhai, C.: Mining Text Data. Springer (2012)
4. Agrawal, K., Aschauer, M., Thonhofer, T., Bala, S., Rogge-Solti, A., Tomsich, N.: Resource Classification from Version Control System Logs. In: EDOC Workshop. vol. September, pp. 249–258 (2016)
5. Akbarinasaji, S., Caglayan, B., Bener, A.: Predicting bug-fixing time: A replication study using an open source software project. J. Syst. Softw. 136, 173–186 (2018)

6. Bala, S., Cabanillas, C., Mendling, J., Rogge-Solti, A., Polleres, A.: Mining Project-Oriented Business Processes. In: Motahari-Nezhad, H.R., Recker, J., Weidlich, M. (eds.) BPM 2015, Innsbruck, Austria. Lecture Notes in Computer Science, vol. 9253, pp. 425–440. Springer (2015)
7. Bala, S., Revoredo, K., de A.R. Gonçalves, J.C., Baião, F., Mendling, J., Santoro, F.: Uncovering the Hidden Co-evolution in the Work History of Software Projects. In: Carmona, J., Engels, G., Kumar, A. (eds.) BPM 2017, Barcelona, Spain. Lecture Notes in Computer Science, vol. 10445, pp. 164–180. Springer (2017)
8. Chen, T.H., Thomas, S.W., Hassan, A.E.: A survey on the use of topic models when mining software repositories. *Empir. Softw. Eng.* 21(5), 1843–1919 (2016)
9. D’Ambros, M., Lanza, M., Lungu, M.: Visualizing co-change information with the evolution radar. *IEEE Trans. Softw. Eng.* 35(5), 720–735 (2009)
10. de A. R. Gonçalves, J.C., Santoro, F.M., Baião, F.A.: Let Me Tell You a Story - On How to Build Process Models. *J. UCS* 17(2), 276–295 (2011)
11. Kindler, E., Rubin, V., Schäfer, W.: Activity Mining for Discovering Software Process Models. *Software Engineering* 79, 175–180 (2006)
12. Kindler, E., Rubin, V., Schäfer, W.: Incremental Workflow Mining Based on Document Versioning Information. In: Li, M., Boehm, B., Osterweil, L. (eds.) *Unifying the Software Process Spectrum*, LNCS, vol. 3840, pp. 287–301. Springer Berlin Heidelberg (2006)
13. Leopold, H.: *Natural Language in Business Process Models - Theoretical Foundations, Techniques, and Applications*, Lecture Notes in Business Information Processing, vol. 168. Springer (2013)
14. Lindberg, A., Berente, N., Gaskin, J.E., Lyytinen, K.: Coordinating Interdependencies in Online Communities: A Study of an Open Source Software Project. *Inf. Syst. Res.* 27(4), 751–772 (2016)
15. Mendling, J., Leopold, H., Pittke, F.: 25 challenges of semantic process modeling. *Int. J. Inf. Syst. Softw. Eng. Big Co.* 1(1), 78–94 (2014)
16. Pinzger, M., Kim, S.: Guest editorial: mining software repositories. *Empir. Softw. Eng.* 21(5), 2033–2034 (2016)
17. Poncin, W., Serebrenik, A., van den Brand, M.: Process mining software repositories. In: *Software Maintenance and Reengineering (CSMR)*, 2011 15th European Conference on. pp. 5–14. IEEE (2011)
18. Richetti, P.H.P., de A. R. Gonçalves, J.C., Baião, F.A., Santoro, F.M.: Analysis of knowledge-intensive processes focused on the communication perspective. In: Carmona, J., Engels, G., Kumar, A. (eds.) BPM 2017, Barcelona, Spain, September 10–15, 2017. LNCS, vol. 10445, pp. 269–285. Springer (2017)
19. Rubin, V., Günther, C.W., van Der Aalst, W.M., Kindler, E., Van Dongen, B.F., Schäfer, W.: Process mining framework for software processes. In: *Software Process Dynamics and Agility*, pp. 169–181. Springer (2007)
20. Ruohonen, J., Hyrynsalmi, S., Leppänen, V.: Time series trends in software evolution. *Journal of Software: Evolution and Process* 27(12), 990–1015 (2015)
21. Thomas, S.W., Hassan, A.E., Blostein, D.: Mining unstructured software repositories. In: *Evol. Softw. Syst.*, pp. 139–162. Springer (2014)
22. Weicheng, Y., Shen, B., Xu, B.: Mining GitHub: Why Commit Stops - Exploring the Relationship between Developer’s Commit Pattern and File Version Evolution. In: Muenchaisri, P., Rothermel, G. (eds.) APSEC 2013, Ratchathewi, Thailand, December 2–5, 2013 - Vol. 2. pp. 165–169. IEEE Computer Society (2013)
23. Zaidman, A., Rompaey, B.V., Demeyer, S., van Deursen, A.: Mining Software Repositories to Study Co-Evolution of Production & Test Code. In: *ICST*. pp. 220–229. IEEE Computer Society (2008)